Control: loops

Control

- conditionals: branching
- **loops**: repetition

1. Meow

```
meow
meow
meow
```

Loop structures in Python

`while` : repeat while a condition is true

`for` : repeat over a sequence (list, string, ...)

`while`: repeat while a condition is true

`while` with a loop counter `i`

```
i = 3

while i > 0:
    print("meow")
    i = i - 1       # or, i -= 1
```

Assignment operators

- `=`: `x = 1`
- `+=`: `x += 1` (same as `x = x + 1`)
- `-=`: `x -= 1` (same as `x = x - 1`)
- `*=`: `x *= 2` (same as `x = x * 2`)
- `/=`: `x /= 2` (same as `x = x / 2`)

...

https://docs.python.org/3/reference/simple_stmts.html#augmented-assignment-statements

`for`: repeat over a sequence (list, string, ...)

```
for i in [0, 1, 2]:    # i is 0, then 1, then 2: meow ×3
    print("meow")

for i in [0, 0, 0]:    # a list of three numbers: still meow ×3
    print("meow")

for i in "abc":        # a sequence of three characters: meow ×3
    print("meow")

for letter in "abc":   # letter is "a", then "b", then "c"
    print(letter)      # a, b, c
```

`range()` to generate a sequence of numbers

```
for i in range(3):          # 0, 1, 2: starts at 0, stops before 3
    print(i)

for i in range(5, 9):       # 5, 6, 7, 8: starts at 5, stops before 9
    print(i)

for _ in range(3):          # _ : a name for a value you won't use
    print("meow")
```

Accumulator: build up a result across a loop

```
total = 0                       # 1. start empty, before the loop
for i in range(1, 6):
    total += i                  # 2. add on each pass; total = total + i
print(total)                    # 3. use it after the loop: 15
```

`i`	`total` after the pass
1	1
2	3
3	6
4	10
5	15

💻 2. Sum and count of even numbers

Requirements:

- Use `input()` to ask for a number `n` and convert it to an integer.
- Print the sum of the even numbers from 1 to `n` (including `n`), then how many even numbers there are.

Expected Outputs:

```
Enter a number: 20
Sum: 110
Count: 10

Enter a number: 7
Sum: 12
Count: 3

Enter a number: 1
Sum: 0
Count: 0
```

3. Interactive meow

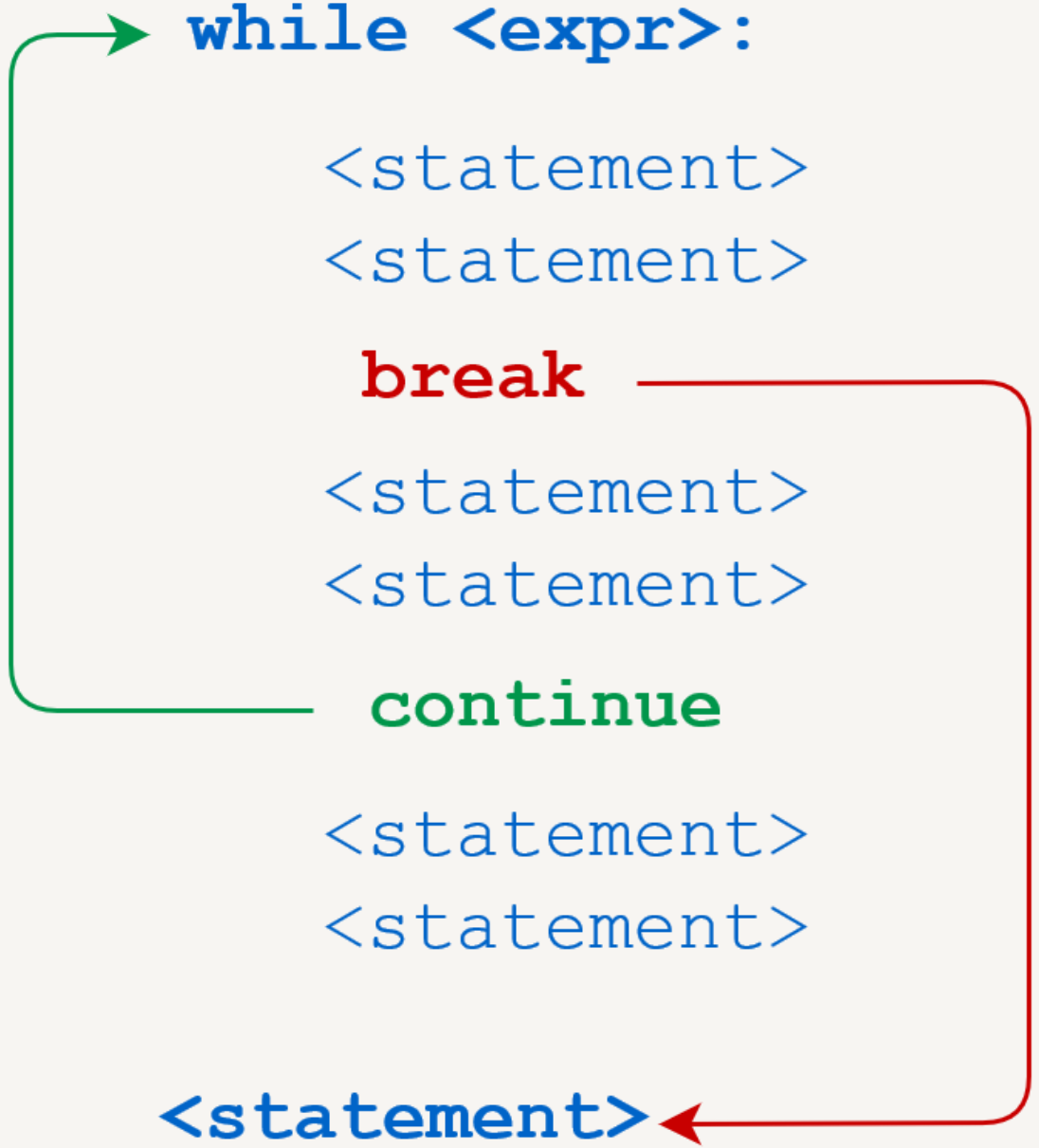
```
Enter a positive number: -3
Enter a positive number: 0
Enter a positive number: 4
meow
meow
meow
meow
```

`while True` for an infinite loop that repeats forever

```
while True:           # the condition is always true
    print("meow")
```

Escape a loop using `break` and `continue`

- `continue` : continue to the next iteration
- `break` : break out of the loop



`continue` skips the rest of this round

```
while True:           # infinite loop
    n = int(input("Enter a positive number: "))

    if n <= 0:        # not positive
        continue      # ask again
    break             # only reached when n > 0

for _ in range(n):
    print("meow")
```

Simpler: `break` when the input is valid

```
while True: # infinite loop
    n = int(input("Enter a positive number: "))

    if n > 0:   # if n is positive
        break   # break out of the loop

for _ in range(n):
    print("meow")
```

Escape a loop using `return`

```
def main():
    n = get_positive_number() # Ask the user to enter a positive number
    meow(n)                   # Print "meow" n times

def get_positive_number():
    while True:
        n = int(input("Enter a positive number: "))
        if n > 0:
            return n        # return the number; this also exits the loop

def meow(n):
    for _ in range(n):
        print("meow")

main()
```

🤔 What gets printed?

```python
# 1
for i in range(5):
    if i == 2:
        continue
    print(i)
```

```python
# 2
for i in range(5):
    if i == 2:
        break
    print(i)
```

```python
# 3
def mystery():
    for i in range(5):
        if i == 2:
            return i
        print(i)
    return -1

print(mystery())
```

```python
# 4
for i in range(5):
    if i % 2 == 0:
        continue
    if i > 2:
        break
    print(i)
```

💻 4. Guess the number

Requirements:

- Start with `secret = 37` (provided).
- Keep asking `Guess: ` until the user guesses the secret:
  - If the guess is too small, print `Too low`; if it's too big, print `Too high`.
- When the guess is right, print `Correct! Number of tries: [tries]` and stop.

Expected Outputs:

```
Guess: 50
Too high
Guess: 25
Too low
Guess: 37
Correct! Number of tries: 3
```

Wrap up

- `while`: repeat while a condition is true; use it when you don't know how many times (e.g., until the input is valid)
- `for`: repeat for each item in a sequence; use it when you know how many times
- `range(start, stop)`: numbers from `start` up to, but not including, `stop`
- `+=`, `-=`, ...: update a variable, e.g., `total += i`
- Accumulator: start a variable before the loop, add to it inside, use it after
- `continue`: skip to the next round; `break`: leave the loop; `return`: leave the loop and the function
- `_`: a loop variable you don't use